

Báo cáo thực hành tìm kiếm — Phần Python (Bài 7–11)

Họ tên: Tạ Nguyễn Tiến Đạt — MSSV: 25022821 — Lớp: K70A-AI1

Phạm vi: báo cáo này ghi phần Python (Bài 7–11). Phần tính tay Bài 1–6 nộp bản viết tay/bút điện tử theo đúng quy định, không thay bằng ảnh đầu ra chương trình.

Môi trường và cách chạy

- Python 3.12.10 (64-bit, Windows), chỉ dùng thư viện chuẩn; không `pip install`.
- Mở thư mục `Greedy_AStar_11Bai` bằng VS Code → Terminal tích hợp, không `cd python`.
- Các lệnh đã dùng (chạy lần lượt, không dán lúc chương trình chờ Enter):

```
python kiem_tra_moi_truong.py
python run_demo.py --algo astar --summary
python -m unittest -v test_cong_khai
python -m unittest -v test_bai_lam
python -m unittest -v test_mau
python chay_bai_lam.py --bai 7
python chay_bai_lam.py --bai 8 --case blocked --summary
python chay_bai_lam.py --bai 10 --case h2 --summary
python chay_bai_lam.py --bai 11 --k 1 --summary
python chay_bai_lam.py --bai 11 --case reopen --summary
```

- Tập cần thiết: `bai_lam.py` (5 hàm nộp), `common.py` (validate, record, make_result, maze_graph, draw_maze), `du_lieu_baitap.py` (dữ liệu đề), `chay_bai_lam.py` (runner), `test_bai_lam.py` (kiểm thử của tôi), `output/bai{7..11}.json` + `output/bai{7..11}_trace.csv` (kết quả chính).
- `test_mau` (22 test) kiểm chương trình mẫu, không kiểm bài nộp. Bài nộp được kiểm bằng `test_cong_khai` (5 test) và `test_bai_lam` (45 test: `DfsTests` 8, `BfsTests` 7, `UcsTests` 7, `GreedyTests` 6, `AstarTests` 9, `ContractTests` 5, `RunnerTests` 3) — đủ 50/50 OK; cộng `test_mau` thì cả thư mục xanh 72/72.
- Chế độ `--step`: dừng TRONG thuật toán sau INIT hoặc một vòng xử lý, bấm Enter ở Terminal (không ở Output/Debug Console); Ctrl+C để dừng.

Bài 7 — DFS

Mô hình. Đồ thị có hướng, danh sách kề có thứ tự, mọi trọng số bằng 1, đỉnh đích `x` không kề ai. Ngăn xếp lưu (đỉnh, cha dự kiến), đỉnh ngăn xếp ở cuối list, ảnh chụp in đảo để phần tử lấy tiếp nằm bên trái.

Dự đoán 3 bước trước khi chạy (viết trước, đối chiếu sau):

bước	phần tử lấy tiếp	ngăn xếp sau xử lý	order
INIT	—	S<-None	[]
1	S	A<-S , B<-S	S
2	A	C<-A , D<-A , B<-S	S, A
3	C	E<-C , D<-A , B<-S	S, A, C

Dự đoán đường $S \rightarrow G$: $S \rightarrow A \rightarrow C \rightarrow E \rightarrow B \rightarrow D \rightarrow G$ (6 cạnh), vì DFS đi hết nhánh bên trái trước khi quay lại nhánh B.

Kết quả $S \rightarrow G$ (python chay_bai_lam.py --bai 7):

```
PATH: ['S', 'A', 'C', 'E', 'B', 'D', 'G']
COST: 6          EDGES: 6
ORDER: ['S', 'A', 'C', 'E', 'B', 'D', 'G']
EXPANDED: 6     SKIPPED: 0     REOPENED: 0     MAX_FRONTIER: 4
```

Kết quả $S \rightarrow X$ (đỉnh không tới được, đồ thị có chu trình $A \rightarrow C \rightarrow A$, $B \rightarrow E \rightarrow B$):

```
PATH: None      COST: inf
ORDER: ['S', 'A', 'C', 'E', 'B', 'D', 'G']
EXPANDED: 7     SKIPPED: 3     MAX_FRONTIER: 4
```

Chương trình kết thúc dù có chu trình: 7 đỉnh tới được đều được thăm đúng một lần, 3 bản ghi trùng đang chờ bị bỏ (SKIP), cuối trace là sự kiện FAIL.

Thay đổi dữ liệu (đảo thứ tự hai cạnh ra của S trên bản sao). Đổi S : [(A,1),(B,1)] thành S : [(B,1),(A,1)] $\rightarrow$ dự đoán DFS lấy B trước:

```
PATH: ['S', 'B', 'D', 'G']      COST: 3
ORDER: ['S', 'B', 'D', 'G']     EXPANDED: 3     SKIPPED: 0     MAX_FRONTIER: 3
```

DFS không bảo đảm ít cạnh nhất: hai lần chạy cho đường 6 và 3 cạnh trên cùng đồ thị, chỉ thứ tự nhánh trong danh sách kề là khác; BFS mới là thuật toán đảm bảo ít bước.

Kiểm thử biên (test_bai_lam.DfsTests, 8 test): đầu trùng đích (path=["S"], expanded=0); khuyên $S \rightarrow S$ + tự khuyên (chấm dứt, skipped=0, order không lặp); bản ghi trùng đang chờ (skipped=1); đồ thị rỗng / thiếu khóa hàng xóm / cạnh song song / danh sách kề không phải list / đầu hoặc đích không có trong đồ thị $\rightarrow$ ValueError; $S \rightarrow X$ kết thúc và trace cuối là FAIL; chế độ --step dừng expanded+skipped+1 lần.

Giải thích thời điểm gán cha. Cha chỉ được chốt khi một bản ghi được pop LẦN ĐẦU HỢP LỆ: dòng visited.add(u); parent[u] = predecessor nằm sau câu lệnh if u in visited: continue, nghĩa là mọi bản ghi cũ hơn của cùng đỉnh đều bị loại trước khi ghi gì vào parent. Nếu gán cha ngay khi đẩy lên ngăn xếp thì đỉnh xuất hiện nhiều lần trong biên (ví dụ B nằm chờ 2 lần) sẽ bị một bản ghi muộn ghi đè cha, dựng đường chạy ngược về đích. Việc chốt trễ cũng làm order chỉ chứa mỗi đỉnh đúng một lần, còn skipped đếm đúng số bản ghi bị bỏ. Đường trả về được dựng từ parent bằng restore_path, không lấy từ order.

Bài 8 — BFS

Mô hình. Mỗi ô đi được tương ứng một đỉnh; ô tường # không sinh đỉnh. Cảnh có hướng R–D–L–U, phí mỗi bước 1. Một ô đi được = một đỉnh vì mọi bước đi đều có cùng chi phí và đúng 4 hướng, nên "số bước ít nhất" trùng với "số cạnh ít nhất" trên đồ thị này.

4 trạng thái hàng đợi đầu (phần tử lấy tiếp ở bên trái):

sự kiện	BEFORE	AFTER
INIT 0,0	[]	['0,0']
EXPAND 0,0	['0,0']	['0,1', '1,0']
EXPAND 0,1	['0,1', '1,0']	['1,0', '0,2']
EXPAND 1,0	['1,0', '0,2']	['0,2', '2,0']

Kết quả mê cung (--bai 8):

```
PATH: [ '0,0', '0,1', '0,2', '1,2', '2,2', '3,2', '3,3', '3,4', '2,4', '1,4' ]
COST: 9      EDGES: 9      EXPANDED: 16      SKIPPED: 0      MAX_FRONTIER: 3
ORDER: 17 đỉnh (gồm đích)
```

Hình ký tự (* là đường trả về, không xoá S/G/tường):

```
S**#. .
.##G.
..***.
...**.
```

Vì sao số ô được xét không bằng độ dài đường. Đường 10 ô (9 bước) nhưng BFS mở rộng 16 ô vì hàng đợi luôn lấy hết một tầng trước khi sang tầng kế: thứ tự lấy bắt đầu 0,0, 0,1, 1,0, 0,2, 2,0, 1,2, 2,1, 3,0, ... — trong đó 1,0, 2,0, 2,1, 3,0, 3,1, 3,5, 2,5 không nằm trên đường lời giải; bốn ô tường thì không bao giờ vào hàng đợi. Số ô xét = số đỉnh vào được hàng đợi đã lấy ra, còn độ dài đường chỉ là số bước trên nhánh cha cuối cùng.

Thay đổi dữ liệu — chặn ô (3,3) (--bai 8 --case blocked , hàng cuối đổi thành "...#..."): cột 3 kín hết 4 hàng (0,3 #, 1,3 #, 2,3 #, 3,3 #) nên vùng bên phải tách hẳn:

```
PATH: None      COST: inf
ORDER: 11 đỉnh [ '0,0', '0,1', '1,0', '0,2', '2,0', '1,2', '2,1', '3,0', '2,2', '3,1', '3,2' ]
EXPANDED: 11     SKIPPED: 0     MAX_FRONTIER: 3
```

Không còn đường tới G : BFS dừng khi hàng đợi rỗng, vùng tới được đúng 11 ô bên trái và dưới.

Kiểm thử biên (BfsTests , 7 test): đầu trùng đích (["G"], 0, 0); mê cung rỗng / không hình chữ nhật / ký tự lạ / thiếu–nhân đôi S/G / water_cost=0 → ValueError ; ô blocked không tới đích; BFS lấy đúng số bước

tối ưu theo oracle liệt kê đường đơn; `h0` của Greedy khác `order` của BFS trên đồ thị nhỏ; đồ thị đầu vào không bị hàm sửa.

Bài 9 — UCS

Dòng cập nhật cả giá và cha (`ucs`, phần nổi lửng):

```
if new_g < best_g.get(v, inf):
    best_g[v] = new_g
    parent[v] = u
    heappush(heap, (new_g, v))
```

Điều kiện `<` chặt: bằng nhau thì giữ cha cũ. Phiếu cũ bị bỏ bằng hai câu: `if g != best_g[u]: skipped += 1; continue` — chạy TRƯỚC `order.append(u)` và TRƯỚC kiểm tra đích.

Bản ghi đầy đủ (`--bai 9`, đọc từ `output/bai9_trace.csv`):

#	event	node	note (trong AFTER)
0	INIT	S	<code>[]</code> → <code>[S(g=0)]</code>
1	EXPAND	S	<code>A:g=8,parent=S; B:g=2,parent=S; G:g=20,parent=S</code>
2	EXPAND	B	<code>A:g=3,parent=B</code> (cải thiện A), <code>D:g=4,parent=B</code>
3	EXPAND	A	<code>C:g=6,parent=A, G:g=12,parent=A</code>
4	EXPAND	D	<code>C:g=5,parent=D</code> (cải thiện C)
5	EXPAND	C	<code>G:g=9,parent=C</code>
6	SKIP	C	<code>old g=6</code> (bản ghi lỗi thời do A tạo)
7	SKIP	A	<code>old g=8</code> (bản ghi lỗi thời của S tạo)
8	GOAL	G	<code>g=9</code>

```
PATH: ['S', 'B', 'D', 'C', 'G']    COST: 9
ORDER: ['S', 'B', 'A', 'D', 'C', 'G']
EXPANDED: 5    SKIPPED: 2    REOPENED: 0    MAX_FRONTIER: 5
```

Hai lần cải thiện: `A` từ 8 xuống 3 khi mở rộng `B`; `C` từ 6 xuống 5 khi mở rộng `D`. Hai bản ghi `C(6)` và `A(8)` vẫn nằm trong heap và bị SKIP ở hai bước liên tiếp trước khi lấy `G(9)`.

Vì sao không dừng ngay khi sinh G. Lần đầu `G` được sinh (từ `S`) chi phí `g=20`; nếu dừng tại đó ta trả đường `S→G` giá 20, trong khi đường thật chỉ 9. UCS phải chờ tới khi `G` được lấy ra với `g` đúng bằng chi phí tối ưu — tức khi không còn phiếu nào trong heap có `g` nhỏ hơn — nên câu kiểm tra đích nằm sau câu bỏ phiếu cũ.

Vì sao không khoá chi phí ngay lần phát hiện đầu. Lần đầu thấy `A` thì `best_g[A]=8`, nhưng đường `S→B→A` chỉ 3; nếu khoá ngay, `C` sẽ được sinh với `g=11` thay vì `6→5`, kéo theo `G` sai. Cập nhật phải mở mỗi khi tìm

được chi phí nhỏ hơn hẳn.

So với BFS trên cùng đồ thị (trọng số gốc):

	đường	cost	expanded
BFS	S→G (1 cạnh)	20	3
UCS	S→B→D→C→G (4 cạnh)	9	5

BFS chọn theo số cạnh nên lấy thẳng cạnh (G,20) ; UCS chọn theo g .

(c) Đổi mọi trọng số về 1 trên bản sao:

	đường	cost	order
UCS đơn vị	S→G	1	S, A, B, G
BFS	S→G	1	S, A, B, G

Chi phí và số cạnh khớp; order chỉ trùng vì hai quy tắc hoà tình cờ cùng chọn tên đỉnh theo thứ tự — đề không đòi hai thuật toán phải có order giống nhau.

Kiểm thử biên (UcsTests , 7 test): khớp dữ liệu chính (đường S→B→D→C→G , cost=9); hai đỉnh cùng g trong heap → hoà theo tên đỉnh (S,A,B,G); hai đường cùng g → giữ cha cũ (S→G giá 3, skipped=0); bản ghi lỗi thời bị bỏ trước khi kiểm tra đích và đích chỉ xuất hiện đúng một lần trong order ; không dừng khi sinh đích (S→G(10) nhưng trả 3); đầu trùng đích & không tới được; trọng số 0 , -1 , inf , NaN , True , "3" → ValueError . Riêng 40 đồ thị ngẫu nhiên khớp oracle liệt kê đường đơn nằm ở ContractTests , mục cuối.

Bài 10 — Greedy

Mô hình. Heap (h, node) , chọn h nhỏ nhất trên TOÀN BỘ biên, đánh dấu khi thêm, giữ cha lần đầu; trọng số không tham gia khoá ưu tiên nhưng được cộng khi tính cost qua path_cost .

Bước rời nhánh. Sau khi xử lý A , hàng xóm duy nhất của A là D với h(D)=7 ; biên đang có B(h=2) , C(h=3) , D(h=7) → thuật toán lấy B với h=2 — một đỉnh KHÔNG phải hàng xóm của A . Phép so 2 < 3 < 7 được lấy trên TOÀN BỘ biên, chứ không lấy trên hàng xóm của đỉnh vừa mở rộng.

Bảng kết quả (--bai 10 , --case h , h2 , h0):

bảng h	đường	cost	order	expanded	max_frontier
h	S→B→E→G	12	S,A,B,E,G	4	3
h2	S→C→G	6	S,C,G	2	3
h0 ≡ 0	S→C→G	6	S,A,B,C,D,E,G	6	3
UCS (đối chiếu)	S→C→G	6	S,A,C,B,D,E,G	6	3

Dự đoán trước khi chạy: tôi đọc nhầm order thành đường lời giải nên viết S→A→B→E→G — sai, vì A không có cạnh ra B . Chương trình trả S→B→E→G (12): parent[G]=E , parent[E]=B , parent[B]=S , nên A xuất hiện

trong `order` nhưng KHÔNG nằm trên đường trả về. Đây là chỗ dễ nhầm giữa `order` và đường lời giải.

Với `h0`, mọi khoá hoà bằng 0 nên thứ tự do tên đỉnh quyết định: `A` lấy trước `B`, `C` lấy trước `D`. `G` được đẩy vào biên khi mở rộng `C` nhưng vẫn phải chờ `D` rồi `E` vì `D < E < G` theo tên; cha của `G` giữ nguyên từ lần phát hiện đầu (`parent[G]=C`) nên đường trả về là `S→C→G` giá 6 — đúng tối ưu nhưng chỉ tình cờ.

Phản ví dụ Greedy `h0` ≠ BFS (đồ thị nhỏ tự tạo):

```
g = {"S": [("B",1),("A",1)], "A": [("C",1)], "B": [("G",1)], "C": [("G",1)], "G": []}
```

	order	path
BFS	S, B, A, G (FIFO)	S→B→G
Greedy <code>h0</code>	S, A, B, C, G (tên đỉnh)	S→B→G

`order` khác nhau dù $h \equiv 0$: BFS lấy theo thời điểm enqueue, Greedy lấy theo hoà khoá trong heap.

Kiểm thử biên (`GreedyTests`, 6 test): khớp dữ liệu chính với `h` (đường `S→B→E→G`, `cost=12`, `order` 5 đỉnh); chọn ứng viên ngoài hàng xóm vừa mở rộng — đúng điều đề cấm "chỉ chọn hàng xóm có `h` nhỏ nhất của đỉnh hiện tại" (sau khi lấy `A` thì lấy `B`, không lấy `D`); `h2` đổi quyết định (`S→C→G`, 2 lần mở rộng); `h0` cho `order` khác BFS; bảng `h` sai (thiếu đỉnh `X`, âm, $h(G) \neq 0$) → `ValueError`; chu trình và đích không tới được (`order=['S','A']`, `path=None`).

Bài 11 — A*

Mô hình phí nước. `rows = ["SwWG", "...", ".##.", "..."]`; phí đi VÀO ô `w` là `k`, vào ô khác đi được là 1, không thu phí ở ô xuất phát; `h` là khoảng cách Manhattan tới `G`. Heap lưu `(f, node, g)` với $f = g + h$.

(b) Bảng so sánh với `k = 5`:

thuật toán	đường	số cạnh	chi phí	expanded	skipped	max_frontier
BFS	0,0 → 0,1 → 0,2 → 0,3	3	11	6	0	4
UCS	0,0 → 1,0 → 1,1 → 1,2 → 1,3 → 0,3	5	5	9	0	5
Greedy	0,0 → 0,1 → 0,2 → 0,3	3	11	3	0	4
A*	0,0 → 1,0 → 1,1 → 1,2 → 1,3 → 0,3	5	5	5	0	5

BFS/Greedy đi hàng trên (3 bước) nhưng mỗi bước vào nước tốn $5 \rightarrow 5+5+1 = 11$; UCS/A* đi vòng xuống hàng 1 $\rightarrow 5$ bước $\times 1 = 5$. Ví dụ này không đủ để kết luận "A* luôn nhanh hơn": nó chỉ cho thấy A* trả chi phí tối ưu với ít lần mở rộng hơn UCS trên BÀI NÀY (5 so với 9).

(c) `k = 1` và `k = 8`, và tính nhất quán của Manhattan:

k	BFS	UCS	Greedy	A*
1	3 (3 cạnh)	3	3	3 (0, 0→0, 1→0, 2→0, 3)
5	11	5	11	5
8	17	5	17	5

Lập luận nhất quán: với hai đỉnh kề u, v , $|h(u) - h(v)| \leq 1$ (Manhattan chỉ đổi 1 đơn vị khi đi một ô) và phí vào ô $c(u,v) \geq 1$ với mọi $k \geq 1$, nên $h(u) \leq 1 + c(u,v) \leq c(u,v) + h(v)$ và tương tự chiều ngược lại $\rightarrow h$ nhất quán, do đó A* luôn tối ưu. Kiểm tự động bằng `test_manhattan_is_consistent` chạy mọi cạnh với $k=5$.

$h = 0$: A* khớp UCS về **path, cost, order, expanded, skipped, max_frontier** cho cả $k = 1, 5, 8$ — cùng quy tắc hoà nên không thể phân biệt (`test_h_zero_matches_ucs` kiểm đúng hệ sáu trường này, trên đồ thị Bài 9).

(d) Đồ thị Bài 5 — mở lại so với không mở lại (`--bai 11 --case reopen`, dữ liệu `REOPEN_GRAPH / REOPEN_H` trong `du_lieu_baitap.py`):

	đường	cost	order	expanded	reopened
<code>reopen=True</code>	<code>S→A→C→G</code>	9	<code>S, B, C, A, C, G</code>	5	1
<code>reopen=False</code>	<code>S→B→C→G</code>	11	<code>S, B, C, A, G</code>	4	0
UCS (đối chiếu)	<code>S→A→C→G</code>	9	—	—	—

Thời điểm cải thiện $g(C)$ — trích trace:

EXPAND B		C: $g=6, f=7, \text{parent}=B$	
EXPAND C		G: $g=11, f=11, \text{parent}=C$	<- C vào CLOSED với $g=6$
EXPAND A		REOPEN C; C: $g=4, f=5, \text{parent}=A$	<- $g(C)$ 6 -> 4, bỏ C khỏi CLOSED
EXPAND C		G: $g=9, f=9, \text{parent}=C$	<- cập nhật cha G, $g(G)$ 11 -> 9
GOAL G		$g=9$	

Bản không mở lại bỏ ứng viên khi $v \in \text{closed}$ TRƯỚC mọi cập nhật nên $g=11$ được trả về; trên cạnh `A→C` có $h(A)=6$ nhưng $c(A,C)+h(C)=1+1=2$, tức $h(A) > c(A,C)+h(C)$ — vi phạm tính nhất quán, đúng điều bài học nêu: h chấp nhận được nhưng không nhất quán thì phải mở lại mới có tối ưu.

Giải thích $g, h, f, \text{discovered}$ và closed . g là chi phí THẬT từ `start` tới đỉnh đang xét, tính bằng cách cộng dần trọng số cạnh nên luôn đúng với đường đã đi; h là ước lượng chi phí còn lại tới đích, do người dùng định nghĩa, không phụ thuộc đường đã đi; $f = g + h$ chỉ là khoá sắp xếp của A, *không phải chi phí thật*. *best_g (bảng g tốt nhất biết tới từng đỉnh) khác g gắn trong từng phiếu: một đỉnh có thể nằm trong heap nhiều bản ghi, chỉ bản trùng best_g khi lấy ra mới hợp lệ, còn lại tính *skipped*. Trong A, `closed` là tập đỉnh ĐÃ MỞ RỘNG với g tốt nhất đã biết — nếu h nhất quán thì không bao giờ cần xét lại; khi h chỉ chấp nhận được, một đường rẻ hơn có thể xuất hiện sau khi đỉnh đã đóng, vì thế `reopen=True` phải bỏ đỉnh khỏi `closed`, cập nhật `best\_g / parent` và đẩy bản mới vào heap (`reopened` đếm số lần đó). Greedy dùng `discovered` đánh dấu ngay khi THÊM vào biên, khác hoàn toàn `closed` (đánh dấu khi MỞ RỘNG) — đó là*

lý do Greedy không bỏ sót nhưng cũng không có bảo đảm tối ưu. `parent` chỉ ghi khi chi phí mới chặt hơn hoặc (với Greedy/BFS/DFS) khi đỉnh mới phát hiện, nên đường trả về lấy từ `parent` chứ không lấy từ `order`.

Kiểm thử biên (`AstarTests`, 9 test): bằng `h` sai (thiếu đỉnh, âm, `inf`, `NaN`, `h(G)≠0`) → `ValueError`; đầu trùng đích; hoà `f` theo tên đỉnh (đỉnh đẩy vào biên trước vẫn bị lấy sau nếu tên lớn hơn: `order = S, A, B, G`); sinh đích nhưng không dừng; `k=1/5/8` khớp oracle; Manhattan nhất quán; `h=0` khớp UCS; mở lại `1` lần (`order` có `C` hai lần) và `reopen=False` trả `11`.

Điều tôi đã tự kiểm chứng

1. **Lỗi đã sửa.** Trong `chay_bai_lam.py`, phần in mê cung cho Bài 8 ban đầu lấy nhầm biến `rows` (dùng `WATER_MAZE` 4×4 cho mê cung 4×6) → `IndexError` tại `draw_maze` vì `"3,5"` vượt cột. Tôi sửa bằng cách đưa `rows` vào dict `info` do `build_case` trả về (`info["rows"]`), xoá hẳn biến cục bộ trùng tên trong `main`. Sau khi sửa: `python chay_bai_lam.py --bai 8` và `--case blocked` đều in đúng hình và ghi file; `python -m unittest test_bai_lam` xanh.
2. **Điểm hiểu rõ hơn (dự đoán sai).** Với Bài 10 tôi đọc nhầm `order` thành đường lời giải nên đoán Greedy trả `S→A→B→E→G`; thực tế là `S→B→E→G` (12), vì `A` không có cạnh ra `B` và `parent[G]=E←B←S` khiến `A` không có con trên đường đi. Nhờ đó tôi kiểm chứng bằng `assert` riêng cho `path` và cho `order` thay vì so chúng với nhau; `test_main_h` `assert` đúng hai giá trị `cost=12` và `order=['S','A','B','E','G']`.
3. **Nhất quán số liệu.** Mọi số trong báo cáo lấy từ `output/bai{7..11}.json` và `*_trace.csv`; `test_trace_counters_match_result` đối chiếu `trace`: số sự kiện `EXPAND` = `expanded`, số `SKIP` = `skipped`, sự kiện đầu là `INIT`, cuối là `GOAL / FAIL` cho cả 5 thuật toán.
4. **Không phụ thuộc đáp án.** UCS/A* được đối chiếu bằng oracle liệt kê mọi đường đơn độc lập (40 đồ thị ngẫu nhiên), BFS bằng oracle số cạnh, mọi đường trả về đều được `path_cost` cộng lại đúng bằng `cost`.

Tài liệu/mã được tham khảo

- `hw2-student.pdf` (đề bài, quy ước thống nhất, hợp đồng hàm và trường kết quả), `HuongDan_CaiDat_Python.pdf` (cách chạy, cách kiểm tra), `README_BAT_DAU.md`.
- `common.py`: chỉ dùng các helper công khai `validate`, `record`, `make_result`, `restore_path`, `path_cost`, `maze_graph`, `draw_maze`, `show_result`; không sửa.
- `*_demo.py` và `run_demo.py`: đọc để HỌC cấu trúc và quy ước trace. `bai_lam.py` KHÔNG import bất kỳ `*_demo.py` nào — kiểm bằng `test_no_import_of_demo_modules` quét các câu lệnh `import / from` trong tệp.
- `du_lieu_baitap.py`: dữ liệu đề bài, giữ nguyên, không sửa.
- Thay đổi do mình thực hiện: viết lại toàn bộ 5 hàm trong `bai_lam.py`, viết `chay_bai_lam.py` (runner + JSON/CSV), viết `test_bai_lam.py` (45 test), bổ sung tên tệp vào `kiem_tra_moi_truong.py`.